VERIQTA

25 REAL-WORLD TERRAFORM PROJECTS

Build, Automate, Secure and Operate Production Infrastructure with Terraform



Plan → Apply → Policy → Drift Check → Operate

- ✓ 25 Practical Projects
- ✓ Real Production Scenarios
- ✓ Architecture & Trade-Offs
- ✓ Security & Compliance
- ✓ Failure & Recovery
- ✓ Senior Engineering Decisions

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

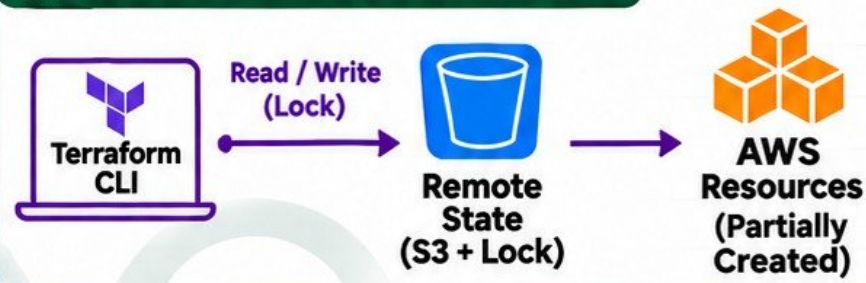
Terraform State Recovery After a Failed Production Deployment

Project 01: Recover infrastructure after a partially successful apply, including remote state inspection, resource reconciliation, state locking, and safe recovery without destroying live resources.

Scenario

A Terraform apply failed midway. Some resources were created, others were not, and the state is now inconsistent with the production environment.

Architecture / State Flow



Recovery Steps (Production-Safe)

1. Identify the failure and scope (terraform apply output, logs).
2. Inspect remote state (terraform state list, show).
3. Verify real resources in cloud (AWS console / CLI) and compare with state.
4. Reconcile state (import missing resources or remove only stale ones$).
5. Re-run a targeted apply (terraform apply -target=...) or a full apply after fix.
6. Verify and unlock state (state confirmed and no pending locks).

Key Commands (Examples)

```
terraform state list                    # Resources in state
terraform state show <id>               # Inspect a resource
terraform import <addr> <id>            # Add existing resource
terraform state rm <addr>               # Remove only stale entry
terraform plan                          # Validate changes
terraform apply                         # Reconcile safely
```

Common Failure Causes

- API timeouts or service limits.
- Partial resource creation.
- State lock left from a crashed run.
- Manual changes made outside Terraform.

Senior Engineer Insight

Never destroy to fix a state issue. First reconcile the state with what actually exists in the cloud, then apply the minimal change.

Verification Checklist

- ✅ State matches real resources
- ✅ Plan shows expected changes only
- ✅ No stale lock
- ✅ Production is healthy

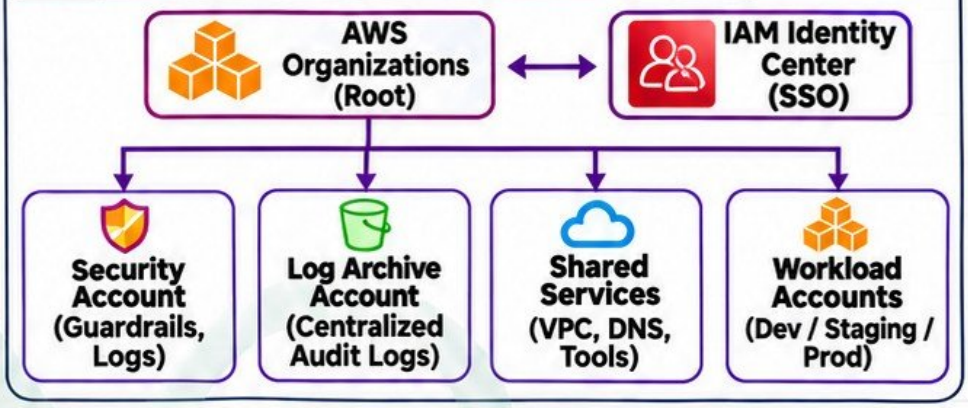
Build a Multi-Account AWS Landing Zone With Terraform

Project 02: Provision account baselines, organizational guardrails, centralized audit logging, IAM Identity Center integration boundaries, and account-specific deployment roles.

Objective

Create a secure, scalable multi-account AWS foundation that enforces governance, security, and operational isolation using Terraform.

Architecture (AWS Landing Zone)



Key Components

- **AWS Organizations:** Central account structure and control.
- **Guardrails:** SCPs (Service Control Policies) to restrict risk.
- **IAM Identity Center:** Central SSO and access management.
- **Centralized Logging:** CloudTrail and AWS Config to a log account.
- **Account Baselines:** Standard networking, security, and tags.
- **Deployment Roles:** Least-privilege roles per account.

Core Terraform Structure

```
landing-zone/
├── modules/ (org, accounts, guardrails, logging)
├── envs/
│   ├── prod/
│   └── nonprod/
├── main.tf (orchestration)
├── variables.tf (account, org, regions)
└── outputs.tf (account IDs, roles, ARNs)
```

Key Benefits

- Strong governance and security boundaries.
- Centralized audit and compliance visibility.
- Consistent, repeatable account provisioning.
- Scales safely as teams and workloads grow.

Senior Engineer Insight

Use AWS Organizations + SCPs to prevent risky actions at the organization level, then apply least-privilege roles and centralized logging for audit, forensics, and compliance.

Verification Checklist

- [x] Accounts created and organized (OU structure)
- [x] SCPs applied and tested (e.g., deny public S3)
- [x] IAM Identity Center users and permission sets working
- [x] CloudTrail and Config logs flowing to log account

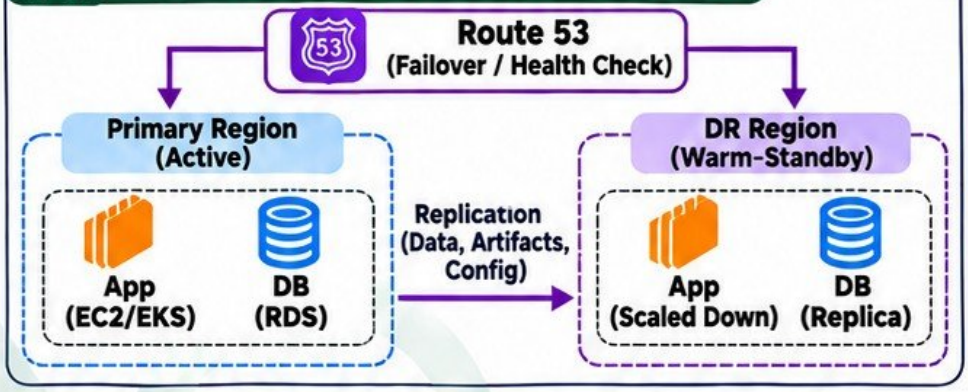
Deploy a Multi-Region Disaster Recovery Environment

Project 03: Build a warm-standby regional environment with replicated application artifacts, database recovery dependencies, Route 53 failover controls, and measurable recovery objectives.

Objective

Ensure business continuity by deploying a multi-region DR setup that can take over quickly with defined RTO and RPO.

Architecture (Warm-Standby DR)



Key Terraform Components

- Multi-region providers (aliases).
- Compute layer (EC2, ASG, or EKS) in both regions.
- Database replication (RDS read replica / cross-region).
- Artifact storage (S3 with cross-region replication).
- Route 53 failover (health checks and DNS records).
- Monitoring and alerts (CloudWatch).

Key Terraform Snippets

```
# Example: Multi-region provider
provider "aws" {
  alias  = "dr"
  region = "eu-west-1   # DR region
}

# Route 53 failover record
resource "aws_route53_record" "app" {
  set_identifier  = "primary"
  failover_routing_policy  {  type = "PRIMARY" }
  health_check_id = aws_route53_health_check.app.id
}
```

Failure Scenarios & Recovery

Scenario	Recovery Action
Primary region outage	Route 53 fails over to DR region.
Database failure	Promote replica and update app.
Artifact missing	Restore from replicated S3 bucket.
Health check false positive	Tune health check (interval, threshold).

DR Targets & Validation

- RTO (Recovery Time Objective): e.g., ≤ 30 minutes
- RPO (Recovery Point Objective): e.g., ≤ 5 minutes
- Verification: Run a DR drill (failover test) and validate application, data, and DNS switch.

Senior Engineer Insight: A warm-standby balances cost and recovery time. Keep the DR environment in sync and test failover regularly.

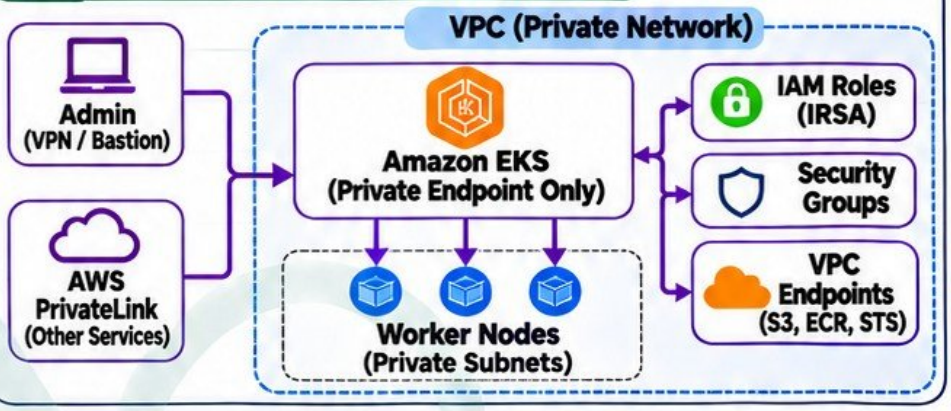
Build a Private EKS Cluster With Zero Public API Exposure

Project 04: Deploy private Kubernetes control-plane access, isolated worker nodes, private endpoints, controlled administrator connectivity, and secure workload identity.

Objective

Provision an Amazon EKS cluster that has no public API endpoint, runs in private subnets, and is accessed securely only from an approved Network (VPN/Bastion/ Direct Connect).

Architecture (Private EKS)



Key Terraform Resources

- **aws_eks_cluster** – EKS with endpoint_private_access = true.
- **aws_eks_node_group** – Worker nodes in private subnets.
- **aws_vpc, aws_subnet** – Isolated networking (no internet exposure).
- **aws_vpc_endpoint** – Private access to ECR, S3, STS, CloudWatch.
- **aws_iam_role + IRSA** – Secure workload identity (least privilege).

Key Terraform Snippet (EKS)

```
resource "aws_eks_cluster" "private" {
  name     = "eks-private-prod"
  role_arn = aws_iam_role.eks_cluster.arn
  vpc_config {
    endpoint_private_access = true
    endpoint_public_access  = false
    subnet_ids = var.private_subnet_ids
  }
```

Common Failure Causes

- Can't access the cluster (wrong network or SG rules).
- ECR image pull failures (missing VPC endpoint).
- Pods can't assume IAM role (IRSA misconfiguration).
- Node launch failure (subnet, route, or IAM issue).

Operational Benefits

- No public control-plane exposure.
- Isolated, private worker nodes.
- Secure access and workload identity,
- Compliant and production-ready.

Senior Engineer Insight

A private EKS reduces attack surface and meets enterprise security requirements, but it adds networking complexity (VPC endpoints, routing, access paths) that must be planned and tested early.

Design a Zero-Trust Network Between Production Workloads

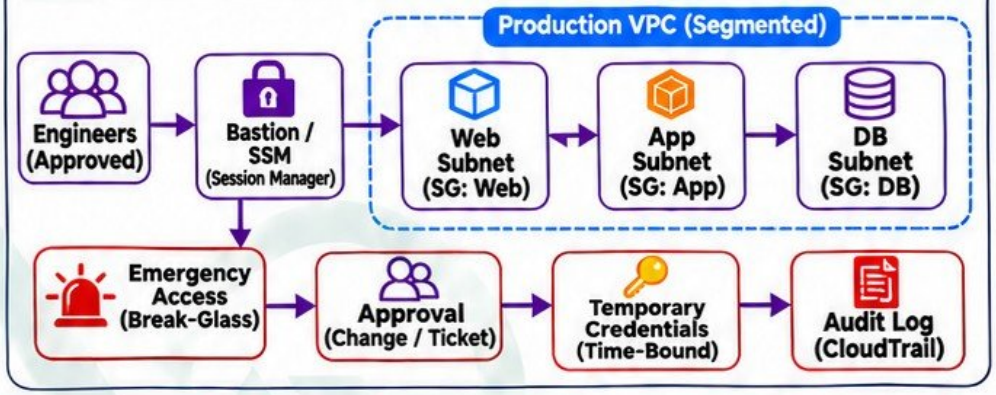
Project 05: Provision segmented application networks, restrictive security groups, private service connectivity, and controlled cross-account access.

Project 06: Build an isolated emergency-access path with temporary credentials, approval controls, and audit evidence.

Objective

Implement a zero-trust architecture that isolates production workloads, allows only verified, least-privilege access, and provides a secure, auditable emergency access path.

Architecture Overview (Zero-Trust + Emergency Access)



Key Design Controls

- **Network Segmentation:** Separate web, app, and database subnets.
- **Restrictive Security Groups:** Allow only required traffic (e.g., Web → App, App → DB).
- **Private Service Connectivity:** Use VPC endpoints / PrivateLink.
- **Cross-Account Access:** IAM roles with least privilege and trust policies.
- **Emergency Access:** Isolated, time-bound access with approvals and audit.

Key Terraform Resources (Examples)

`aws_vpc`, `aws_subnet` – Network and segmentation.
`aws_security_group` – Restrictive traffic rules.
`aws_vpc_endpoint` – Private service access (S3, ECR, etc.).
`aws_iam_role` – Cross-account and emergency roles.
`aws_iam_policy` – Least-privilege permissions.
`aws_cloudtrail` – Audit and access evidence.
`aws_ssm_session_manager` – Secure admin access (no open SSH).

Risks & Failure Considerations

- Overly broad security group rules (breaks zero-trust).
- Unapproved or permanent break-glass access.
- Missing audit logs or access traces.
- Cross-account role misconfiguration (excess permissions).

Benefits

- Strong workload isolation.
- Reduced attack surface.
- Controlled and auditable access.
- Compliant with security best practices.

Senior Engineer Insight

Zero-trust is not just about network isolation. It's a combination of segmentation, least privilege, verified identity, and audited access — including a safe, controlled break-glass path for real-world incidents.

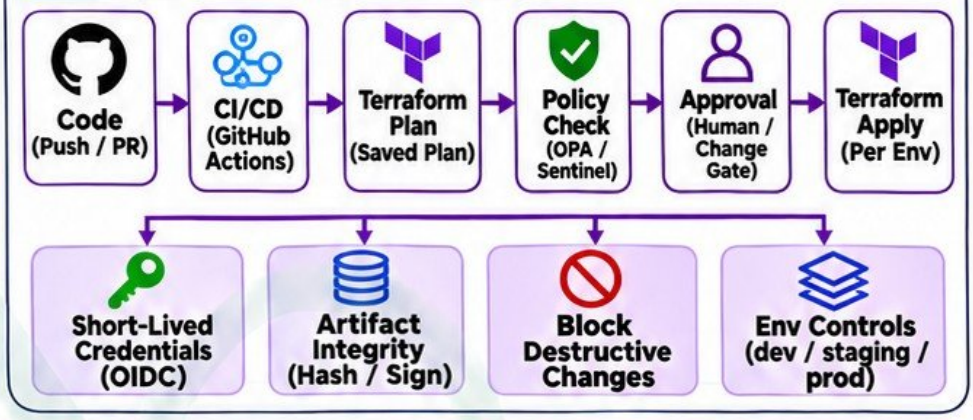
Engineer a Terraform Deployment Pipeline That Rejects Dangerous Changes

Project 07: Implement saved-plan approval, policy checks, destructive-change detection, short-lived credentials, artifact integrity, and environment-specific deployment controls.

Objective

Build a secure, auditable Terraform pipeline that creates a plan, enforces policies, blocks risky changes (e.g., deletes), uses short-lived credentials, and deploys safely per environment (dev, staging, prod).

Pipeline Architecture



Key Controls Implemented

- **Saved Plan:** Generate and store terraform plan for review.
- **Policy Checks:** Enforce rules (OPA/Sentinel) e.g., no public S3.
- **Destructive-Change Detection:** Fail pipeline for unexpected deletes.
- **Short-Lived Credentials:** Use OIDC (GitHub → AWS IAM role).
- **Artifact Integrity:** Verify plan integrity (hash/sign) before apply.
- **Environment Controls:** Separate workflows, approvals, and permissions per environment.

Example: Policy Check (Terraform + OPA)

```
# OPA policy: block destructive changes
package terraform.guardrails

deny[msg] {
  input.resource_changes[_].change.actions == ["delete"]
  msg := "Deletion not allowed without approval"
}
```

When the Pipeline Should Reject

- Plan contains delete or replace of critical resources.
- Resource violates policy (e.g., public access, untagged).
- Missing mandatory approvals for production.
- Plan hash mismatch (tampered artifact).
- Wrong branch or environment target.

Business & Operational Benefits

- Prevents accidental production outages.
- Enforces security and compliance.
- Creates a clear change audit trail.
- Reduces blast radius of risky changes.
- Supports safe, repeatable deployments.

Senior Engineer Insight

Treat Terraform changes like code releases: plan → validate → approve → apply. The goal is not just automation, but controlled, reversible, and auditable infrastructure change.

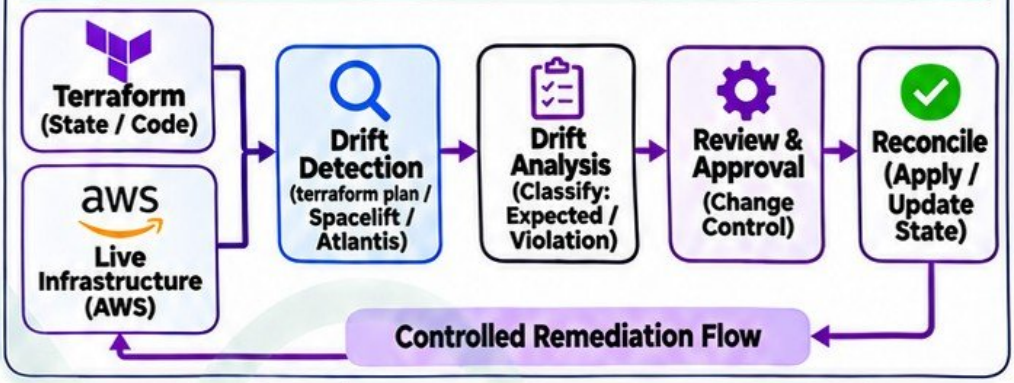
Detect Infrastructure Drift and Reconcile Unauthorized Changes

Project 08: Detect console modifications, identify security-sensitive drift, distinguish expected external changes from configuration violations, and route remediation through controlled reviews.

Objective

Continuously detect and manage infrastructure drift, identify unauthorized changes, and reconcile the environment with approved Terraform configurations.

Architecture Overview (Drift Detection & Reconciliation)



Key Activities

- Run scheduled Terraform plan against the live environment.
- Detect and list configuration drift.
- Classify drift as expected (e.g., autoscaling, tags) or violation (e.g., open security group, public S3).
- Route violations for review and approval.
- Reconcile by updating code or state (import/align) and apply safely.

Key Terraform Commands

```
# Detect drift (no changes applied)
terraform plan -detailed-exitcode

# Refresh state and compare with live resources
terraform plan -refresh-only

# Update state to match an existing resource
terraform import aws_s3_bucket.logs my-audit-logs

# Reconcile after approved changes
terraform apply
```

Drift Classification Examples

Change Detected	Type	Action
Security group opened to 0.0.0.0/0	Violation	Revert via Terraform
AutoScaling desired capacity change	Expected	Document and allow
S3 bucket encryption disabled	Violation	Revert and add policy
New tag added by monitoring tool	Expected	Keep and update code

Tools & Integrations (Options)

- Terraform (plan, import, apply)
- AWS CloudTrail (who changed what)
- Spacelift / Atlantis (drift detection & review)
- AWS Config (compliance and change history)
- GitHub Actions (scheduled drift checks)

Benefits

- Early detection of unauthorized changes.
- Stronger security and compliance.
- Controlled and auditable remediation.
- Reduced configuration mismatch and risk.

Senior Engineer Insight

Drift will always happen in real systems. The goal is not to eliminate it, but to detect it quickly, classify it correctly, and reconcile it through a controlled, auditable process.

Build a Multi-Tenant SaaS Infrastructure Isolation Model

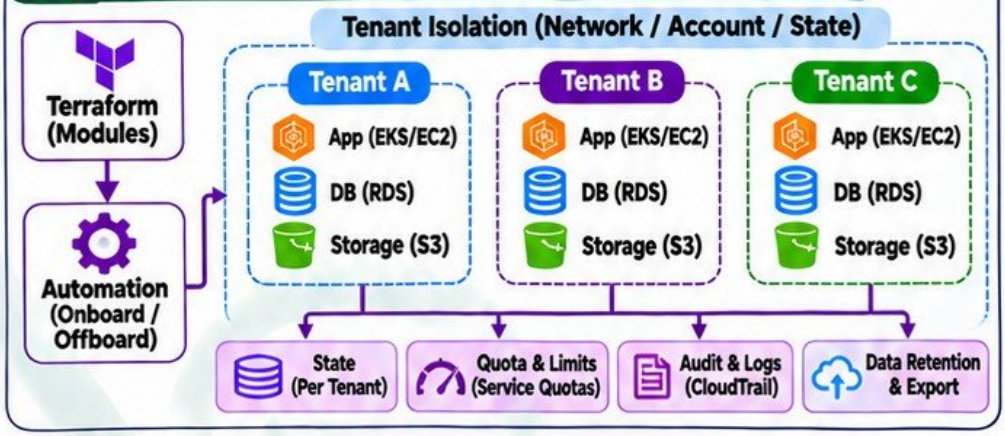
Project 09: Provision tenant-specific infrastructure with reusable modules, account or network boundaries, quota controls, and isolated deployment state.

Project 10: Build automated tenant provisioning and offboarding with retention, export, and deletion safeguards.

Objective

Design a secure, scalable, and operationally efficient multi-tenant SaaS infrastructure that isolates tenants, automates lifecycle management, and protects tenant data and resources.

Architecture Overview (Multi-Tenant SaaS)



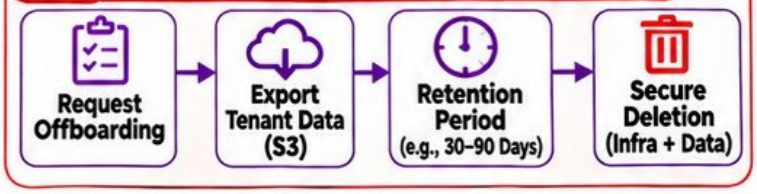
Key Design Controls

- **Tenant Isolation:** Separate accounts or VPCs/subnets per tenant.
- **Reusable Modules:** Standard, versioned Terraform modules for all tenants.
- **Isolated State:** Dedicated, encrypted remote state per tenant (e.g., S3 + DynamoDB).
- **Quota Controls:** Enforce resource limits (EC2, storage, DB, etc.).
- **Automated Lifecycle:** Self-service provisioning and safe offboarding.

Key Terraform Snippets

```
# Tenant module with isolation and limits
module "tenant" {
  source          = "./modules/tenant"
  tenant_id       = var.tenant_id
  vpc_cidr        = var.vpc_cidr
  instance_quota  = var.instance_quota
  db_size         = var.db_size
}
# Isolated state (per tenant)
backend "s3" {
  bucket         = "veriqta-terraform-state-tenant-a"
  key            = "tenant-a/terraform.tfstate"
  dynamodb_table = "terraform-locks"
}
```

Tenant Offboarding Safeguards



Benefits

- Strong tenant isolation and data security.
- Faster, consistent tenant provisioning.
- Controlled resource usage and cost.
- Safe, auditable offboarding with retention.

Senior Engineer Insight

Multi-tenancy is not just about separating data — it's about isolation, automation, compliance, and a full tenant lifecycle.

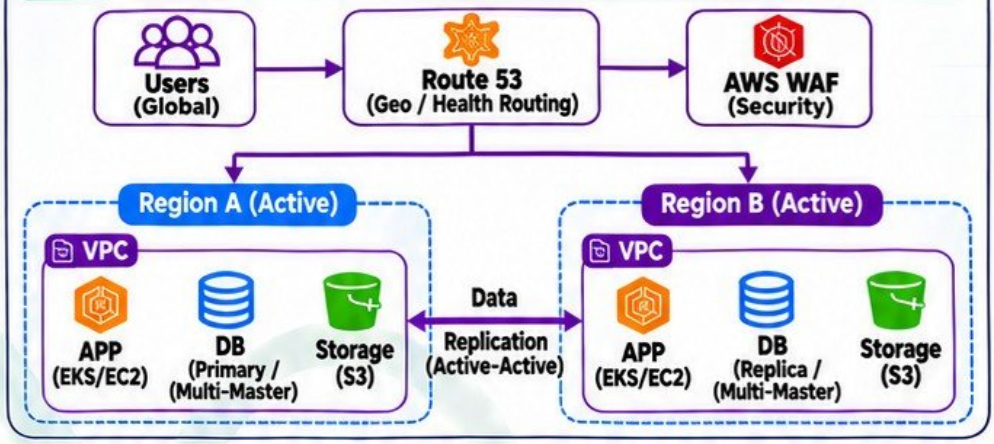
Provision an Active-Active Regional Application Platform

Project 11: Build independent regional compute stacks, global traffic distribution, health-based routing, and failure-domain isolation. Address data consistency and regional dependency risks.

Objective

Deploy a highly available, active-active application platform across multiple regions with global routing, independent stacks, and a resilient data strategy.

Architecture Overview (Active-Active, Multi-Region)



Key Design Components

- **Independent Regional Stacks:** Isolated, identical deployments (per region).
- **Global Traffic Distribution:** Route 53 geo/latency + health checks.
- **Failure-Domain Isolation:** Region, AZ, and service boundaries.
- **Data Consistency:** Multi-region replication (e.g., Aurora Global DB / DynamoDB Global Tables).
- **Dependency Risk Control:** Minimize cross-region calls and use resilient service patterns.

Key Terraform Resources (Examples)

```
# Multi-region provider setup
provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"
}
provider "aws" {
  alias  = "eu_west_1"
  region = "eu-west-1"
}
module "app_region_a" {
  source  = "./modules/region-stack"
  providers = { aws = aws.us_east_1 }
module "app_region_b" {
  source  = "./modules/region-stack"
  providers = { aws = aws.eu_west_1 }
```

Key Risks & Mitigations

Risk	Mitigation
Data inconsistency	Use multi-region DB (Aurora/DynamoDB)
Regional dependency	Design for region autonomy
Traffic routing failure	Health checks with automatic failover
Higher cost	Right-size and use autoscaling
Complex operations	Standardized Terraform modules

Benefits

- High availability (no single-region failure).
- Lower user latency (global routing).
- Independent and isolated deployments.
- Resilient, scalable, and production-ready.

Senior Engineer Insight

Active-active gives the highest availability and user experience, but it requires a solid data strategy, strict regional isolation, and careful management of cross-region dependencies.

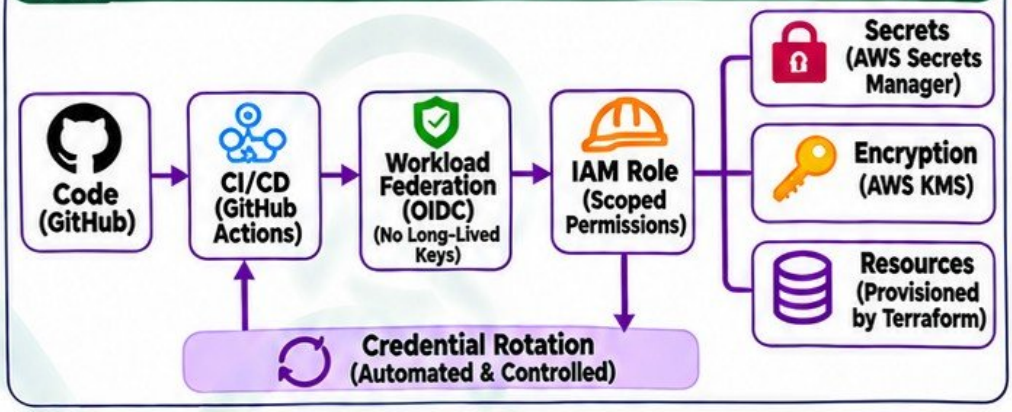
Engineer a Secure Terraform Secrets and Identity Architecture

Project 12: Use workload federation, scoped IAM roles, secret references, encryption, and controlled credential rotation without embedding long-lived cloud credentials in pipelines.

Objective

Build a secure authentication and secrets model for Terraform that eliminates long-lived credentials, enforces least privilege, and protects sensitive configuration and data.

Architecture Overview (Identity & Secrets Flow)



Key Design Controls

- **Workload Federation:** Use OIDC (GitHub Actions → AWS) instead of static access keys.
- **Scoped IAM Roles:** Least-privilege permissions per environment (dev, staging, prod).
- **Secret References:** Store and fetch secrets from AWS Secrets Manager (or SSM Parameter Store).
- **Encryption:** Use KMS for secrets, state, and sensitive data at rest.
- **Credential Rotation:** Automated rotation and short-lived, time-bound credentials.

Key Terraform Implementation

```
# OIDC provider (GitHub → AWS)
resource "aws_iam_openid_connect_provider" "github" {
  url = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
}

# Role for Terraform (scoped access)
resource "aws_iam_role" "terraform" {
  name = "terraform-deploy-role"
  assume_role_policy = data.aws_iam_policy_document.oidc.json
}

# Reference a secret (no hardcoded value)
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "prod/app/db"
}
```

Security Anti-Patterns to Avoid

- Hardcoding cloud access keys in code or pipelines.
- Using overly broad IAM permissions (e.g., Admin).
- Storing secrets in plain text (e.g., tfvars, Git).
- Long-lived credentials without rotation.

Benefits

- No long-lived credentials in pipelines.
- Least-privilege and environment isolation.
- Encrypted and centrally managed secrets.
- Auditable, compliant, and production-ready.

Senior Engineer Insight

Terraform security is not just about infrastructure code — it's about identity, secrets, and access control. Use short-lived, federated access, strict IAM scope, and encrypted secret management.

Build an Ephemeral Preview Environment Factory

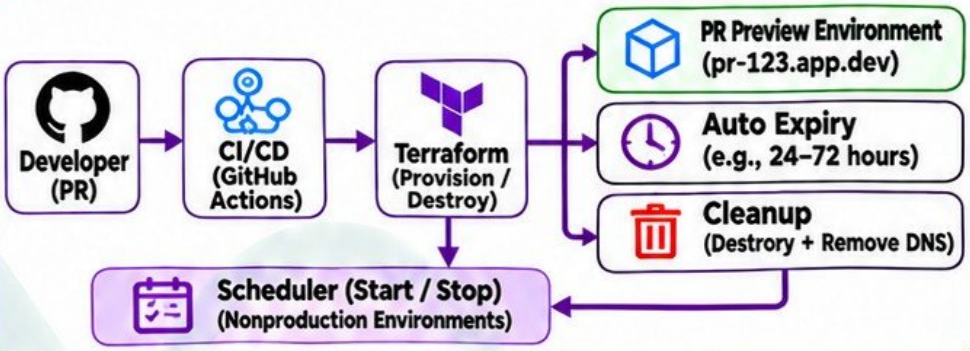
Project 13: Provision pull-request environments with isolated state, temporary DNS, cost ceilings, automatic expiry, and cleanup safeguards.

Project 14: Build a scheduled environment shutdown and restoration system for nonproduction infrastructure.

Objective

Automate the creation, management, and cleanup of temporary and nonproduction environments to accelerate testing, reduce cost, and keep infrastructure clean and controlled.

Architecture Overview (Preview & Lifecycle Automation)



Key Capabilities

- Ephemeral Environments: On-demand PR environments with isolated state.
- Temporary DNS: Automatic, unique subdomain per PR (e.g., pr-123.dev).
- Cost Ceilings: Limits and alerts to prevent cost overrun.
- Automatic Expiry: Time-based destruction (TTL).
- Scheduled Control: Start/stop nonproduction environments (e.g., nights/weekends) and restore on demand.

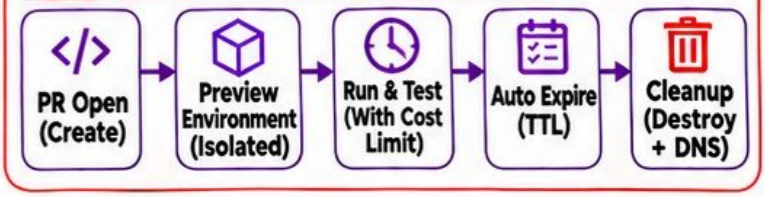
Example: Automation Workflow (GitHub Actions)

```
# Create preview environment (on PR open)
- name: Provision Preview
  run: terraform apply -auto-approve\
       -var="env=pr-${{ github.event.number }}"

# Auto-destroy (on PR close or TTL)
- name: Destroy Preview
  run: terraform  destroy -auto-approve \
       -var="env=pr-${{ github.event.number }}"

# Scheduled stop/start (nonproduction)
- name: Stop nonproduction (nightly)
  run: terraform apply -var="action=stop"
- name: Start nonproduction (weekdays)
  run: terraform apply -var="action=start"
```

Environment Lifecycle



Benefits

- Faster and safer feature validation.
- Lower infrastructure cost.
- Clean, consistent, and isolated environments.
- Automated lifecycle with minimal manual work.

Senior Engineer Insight

Ephemeral environments give every change a safe, real-world test space while keeping cost and infrastructure under control.

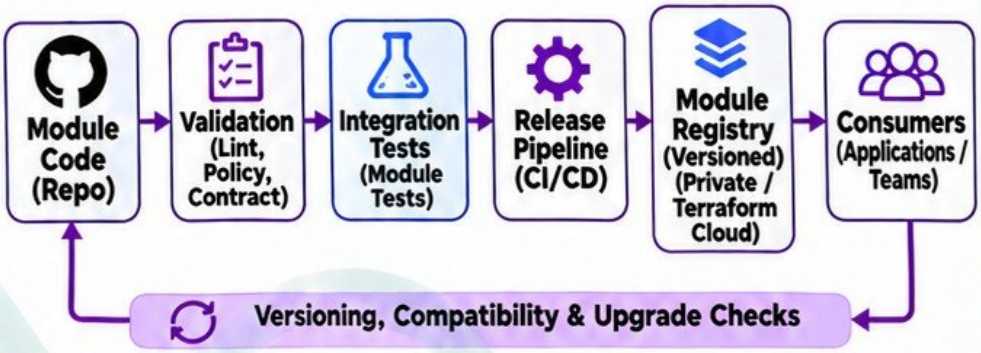
Build a Production-Grade Terraform Module Registry

Project 15: Develop versioned infrastructure modules with contracts, validation, integration testing, release pipelines, compatibility policies, and automated consumer upgrade checks.

Objective

Create a reliable, scalable, and production-ready Terraform module registry that delivers reusable, versioned, tested, and backwards-compatible infrastructure modules.

Architecture Overview (Module Registry & Lifecycle)



Key Capabilities

- **Versioned Modules:** Semantic versioning (e.g., v1.2.0).
- **Contracts:** Defined inputs, outputs, and usage expectations.
- **Validation:** Linting, policy checks (e.g., tflint, OPA/Sentinel).
- **Integration Testing:** Automated tests with real or mocked resources.
- **Release Pipeline:** Build, test, tag, and publish to registry.
- **Compatibility Policy:** Backward compatibility and deprecation process.
- **Automated Upgrade Checks:** Notify consumers of new compatible versions.

Example: Module Structure & Release

```
terraform-modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    ├── versions.tf
    ├── README.md
    ├── examples/
    └── tests/
        └── vpc_test.go
```

Versioning (versions.tf)

```
terraform {
  required_version = ">= 1.6.0"
}
```

Module release (tag & publish)

```
git tag v1.2.0
git push origin v1.2.0
terraform login
terraform module publish \
  veriqta/vpc/aws v1.2.0
```

Module Quality Gates

Gate	Purpose	Tool (Example)
Lint & Format	Code quality and style	tflint, terraform fmt
Policy Check	Enforce standards	OPA / Sentinel
Integration Test	Validate real behavior	Terratest
Versioning	Controlled releases	Git tags + CI/CD
Compatibility	Prevent breaking changes	terraform validate + tests

Benefits

- Reusable, tested, and trusted modules.
- Faster and more consistent deployments.
- Lower risk of configuration errors.
- Clear versioning and upgrade path.
- Scalable across teams and environments.

Senior Engineer Insight

A module registry turns infrastructure code into a product — versioned, tested, and safe to reuse at scale across teams.

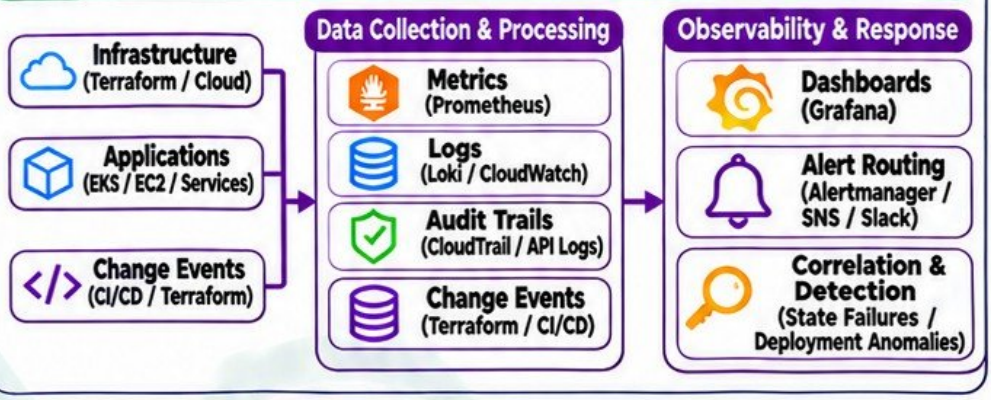
Deploy an Infrastructure Observability and Audit Platform

Project 16: Provision centralized metrics, logs, audit trails, alert routing, and infrastructure change correlation. Detect state backend failures and deployment anomalies.

Objective

Build a centralized observability and audit platform for infrastructure that provides real-time visibility, change tracking, alerting, and early detection of state and deployment issues.

Architecture Overview (Observability, Audit & Correlation)



Key Capabilities

- **Centralized Metrics:** Infrastructure and application health (Prometheus).
- **Centralized Logs:** Aggregated and searchable (Loki / CloudWatch).
- **Audit Trails:** API and config changes (CloudTrail).
- **Change Correlation:** Link infra changes (Terraform / CI/CD) to metrics and logs.
- **State Backend Monitoring:** Detect S3/DynamoDB state failures (e.g., lock, access, latency).
- **Deployment Anomaly Detection:** Identify failed, stuck, or abnormal deployments and alert automatically.

Key Terraform & Monitoring Examples

```
# Terraform: S3 backend (state + lock)
terraform {
  backend "s3" {
    bucket    = "veriqta-terraform-state"
    key       = "infra/prod/terraform.tfstate"
    region    = "us-east-1"
    dynamodb_table = "terraform-locks"
  }
}
# Terraform: Create Prometheus / Grafana (EKS)
module "observability" {
  source  = "./modules/observability"
  enable_prometheus  = true
  enable_grafana     = true
  alert_email        = "sre@veriqta.com"
}
```

Key Alerts & Use Cases

Alert / Detection	Example Trigger
State backend failure	S3 access error, DynamoDB lock timeout
Deployment anomaly	Terraform apply failed or timed out
Unauthorized change	API config change outside CI/CD
Infrastructure health	High CPU, error rate, or node down
Change correlation	Link spike to recent Terraform change

Benefits

- Full visibility into infrastructure and changes.
- Faster detection and troubleshooting.
- Early warning for state and deployment issues.
- Auditable and compliant operations.
- Improved reliability and operational control.

Senior Engineer Insight

Observability plus audit and change correlation turns infrastructure into a measurable, traceable, and resilient system.

Build a Cost-Aware Kubernetes Capacity Platform

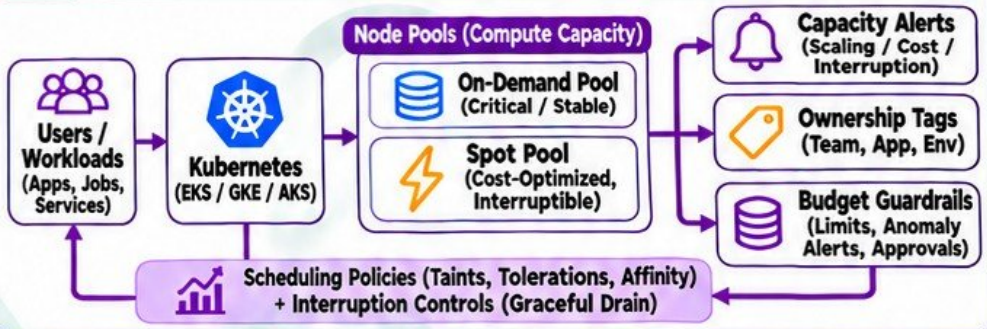
Project 17: Provision workload-specific node pools, Spot and On-Demand capacity, interruption controls, scheduling boundaries, and capacity-related alerting.

Project 18: Build infrastructure budget guardrails with mandatory ownership tags, anomaly alerts, and approved resource classes.

Objective

Build a Kubernetes capacity and cost management platform that optimizes compute usage, controls spend, and enforces governance with clear ownership and alerting.

Architecture Overview (Kubernetes Capacity & Cost Control)



Key Capabilities

- **Workload-Specific Node Pools:** Separate pools for critical, standard, and batch workloads.
- **Hybrid Capacity:** Use Spot for cost savings and On-Demand for reliability.
- **Interruption Controls:** Handle Spot terminations with graceful drain and rescheduling.
- **Scheduling Boundaries:** Enforce placement with taints, tolerations, and affinity rules.
- **Capacity Alerting:** Monitor utilization, scaling, and capacity risks.
- **Budget Guardrails:** Enforce ownership tags, spend limits, and approved resource classes.

Example: Key Kubernetes & Policy Snippets

```
# Node pool with Spot capacity (EKS example)
eksctl create nodegroup \
  --cluster veriqta-eks \
  --name spot-pool
  --node-type m6a.large
  --capacity-type SPOT\
  --nodes 2 --nodes-max 10
```

```
# Pod with scheduling rules (taint/toleration + affinity)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: batch-worker
spec:
  template:
    spec:
      tolerations:
      - key: "workload"
        operator: "Equal"
        value: "batch"
        effect: "NoSchedule"
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
              -- key: "nodepool"
              - operator: "In"
                values: ["spot"]
```

Cost Guardrails & Governance

Control	Implementation Example
Ownership Tagging	team, app, environment, cost-center
Budget Limit	AWS Budgets or Kubecost (e.g., $1,000/mo)
Anomaly Alert	> 20% daily or weekly cost increase
Approved Classes	Allowlist (e.g., m6a.*, c6a.* only)
Capacity Policy	Min/Max nodes, Spot vs On-Demand ratio

Benefits

- Lower infrastructure cost (Spot + right-sizing).
- Higher availability for critical workloads.
- Controlled and predictable cloud spend.
- Clear cost ownership and accountability.
- Scalable and production-ready Kubernetes platform.

Senior Engineer Insight

A cost-aware Kubernetes platform balances performance, reliability, and cost control through smart capacity, policies, and strong governance.

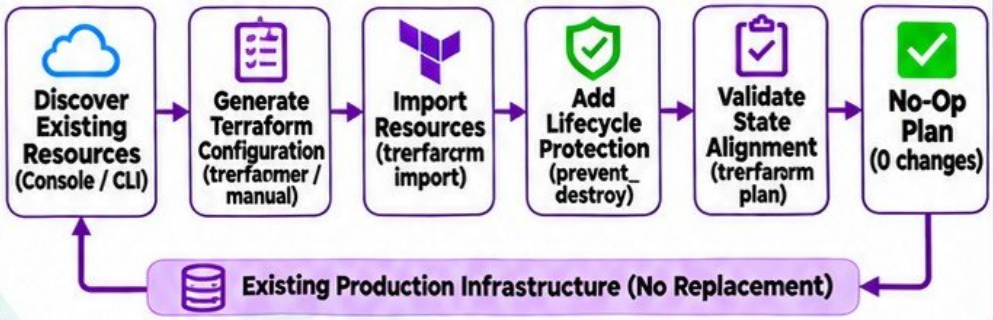
Migrate Live Infrastructure Into Terraform Without Replacement

Project 19: Import existing production resources, generate and review configuration, establish lifecycle protection, validate state alignment, and achieve a no-op plan before future changes.

Objective

Bring existing production infrastructure under Terraform management without replacing or disrupting any live resources.

Migration Workflow (Import to No-Op)



Key Capabilities

- Resource Discovery: Identify and inventory live production resources.
- Configuration Generation: Create Terraform code that matches existing resources.
- Resource Import: Bring resources into Terraform state without recreating them.
- Lifecycle Protection: Use prevent_ destroy and guardrails for critical resources.
- State Validation: Ensure Terraform state matches the actual infrastructure.
- No-Op Confirmation: Run terraform plan and confirm 0 changes before proceeding.

Example: Import & Validate (AWS)

```
# 1. Create resource configuration (example: S3)
resource "aws_s3_bucket" "app" {
  bucket = "veriqta-prod-app"
  lifecycle {
    prevent_destroy = true
  }
}

# 2. Import the existing bucket into state
terraform import aws_s3_bucket.app veriqta-prod-app

# 3. Validate state and confirm no changes
terraform plan
```

No changes. Your infrastructure matches the configuration.

Migration Checklist

Step	Purpose
Inventory resources	Know what exists in production
Create Terraform config	Define matching definitions
Import resources	Add live resources to state
Add lifecycle rules	Protect critical resources
Validate (plan)	Confirm 0 changes (no replacement)

Benefits

- Brings production under IaC safely.
- No downtime or resource replacement.
- Controlled and auditable future changes.
- Built-in protection for critical resources.
- Better visibility and long-term management.

Senior Engineer Insight

Import, align, and protect — the right Terraform migration keeps production stable today and fully manageable tomorrow.

Build a Cross-Account Private Connectivity Platform

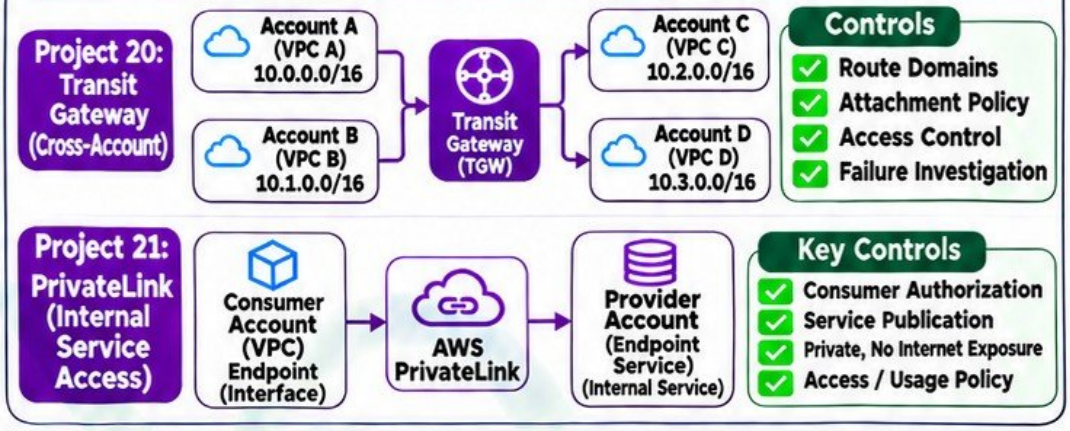
Project 20: Provision a Transit Gateway architecture with explicit route-domain separation, attachment controls, and failure investigation paths.

Project 21: Build a PrivateLink-based internal service publication model with consumer authorization and reduced network exposure.

Objective

Build secure, scalable, and controlled private network connectivity across accounts using Transit Gateway and PrivateLink to enable internal service access with minimal exposure.

Architecture Overview (TGW + PrivateLink)



Key Capabilities

- Transit Gateway Architecture: Central hub for cross-account VPC connectivity.
- Route-Domain Separation: Isolate workloads (e.g., prod, staging, shared).
- Attachment Controls: Govern which accounts/VPCs can connect.
- Failure Investigation: Trace routing and connectivity issues quickly.
- PrivateLink Service Publication: Expose internal services privately.
- Consumer Authorization: Allow only approved consumers to access services.
- Reduced Exposure: Keep services private within AWS.

Example: TGW & PrivateLink (AWS)

```
# Project 20: Create Transit Gateway and attach VPC
resource "aws_ec2_transit_gateway" "tgw" {
  description = "veriqta-shared-tgw"
}
resource "aws_ec2_transit_gateway_vpc_attachment" "vpc_a" {
  transit_gateway_id = aws_ec2_transit_gateway.tgw.id
  vpc_id             = var.vpc_a_id
}

# Project 21: Publish and consume a service using PrivateLink
resource "aws_vpc_endpoint_service" "svc" {
  acceptance_required        = true
  network_load_balancer_arns = [var.nlb_arn]
}
resource "aws_vpc_endpoint" "consumer" {
  service_name      = aws_vpc_endpoint_service.svc.service_name
  vpc_id            = var.consumer_vpc_id
  vpc_endpoint_type = "Interface"
}
```

Key Outcomes

- Secure cross-account connectivity via TGW.
- Privately published internal services via PrivateLink.
- Controlled, auditable, and isolated network access.

Benefits

- Secure, scalable, and cost-efficient connectivity.
- Lower network exposure and attack surface.
- Clear isolation between environments or domains.
- Faster troubleshooting and failure investigation.

Senior Engineer Insight

Use Transit Gateway for scalable, multi-account networking and PrivateLink for secure, private internal service access.

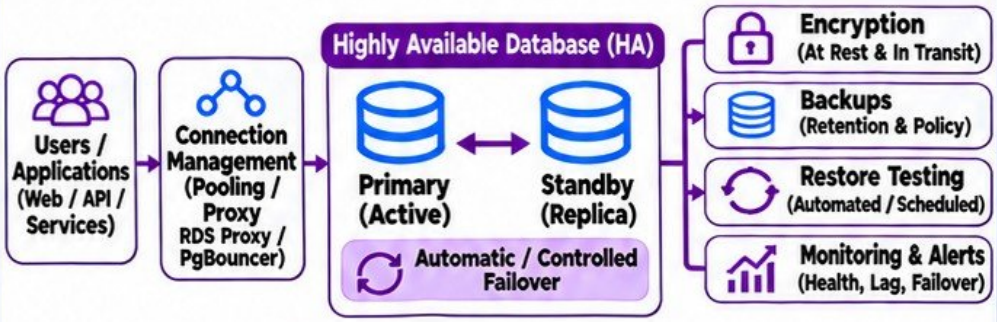
Engineer a Production Database Resilience Platform

Project 22: Provision a highly available relational database environment with encryption, backup retention, restore testing infrastructure, connection management, and controlled failover exercises.

Objective

Build a production-grade, highly available database platform that protects data, ensures recovery, maintains stable connections, and reduces downtime through tested failover.

Architecture Overview (Database Resilience)



Key Capabilities

- **High Availability:** Primary and replica with automated or controlled failover.
- **Encryption:** Data at rest and in transit (TLS).
- **Backup & Retention:** Automated backups with defined retention (e.g., 7/30/90 days).
- **Restore Testing:** Regular, verifiable restore exercises in an isolated environment.
- **Connection Management:** Use a proxy or pooler for stable, scalable connections.
- **Failover Exercises:** Controlled testing to validate recovery time and application behavior.

Example: Database (AWS RDS + CLI)

```
# Create a Multi-AZ database (PostgreSQL example)
aws rds create-db-instance \
    --db-instance-identifier veriqta-prod-db \
    --engine postgres \
    --db-instance-class db.t4g.medium
    --allocated-storage 100 \
    --multi-az \
    --storage-encrypted \
    --backup-retention-period 30

# Test a restore to a new environment
aws rds restore-db-instance-to-point-in-time \
    --source-db-instance-identifier veriqta-prod-db
    --target-db-instance-identifier veriqta-restore-test
    --restore-time 2026-11-10T12:00:00Z

# (Controlled) Failover for testing (Multi-AZ)
aws rds reboot-db-instance \
    --db-instance-identifier veriqta-prod-db
    --force-failover
```

Resilience Checklist

Area	How to Validate
✓ High Availability	Verify replica and failover works.
✓ Encryption	Confirm encryption at rest and in transit.
✓ Backups	Check backup success and retention.
✓ Restore Test	Restore to a test instance and validate data.
✓ Connections	Test via proxy/pooler under load.
✓ Failover Exercise	Run a controlled failover and verify uptime.

Benefits

- Higher availability and business continuity.
- Strong data protection and compliance.
- Faster and proven recovery (tested restores).
- Stable and scalable application connections.
- Reduced risk of data loss and downtime.

Senior Engineer Insight

A resilient database platform is not just about backups — it's about tested recovery, secure access, and proven failover.

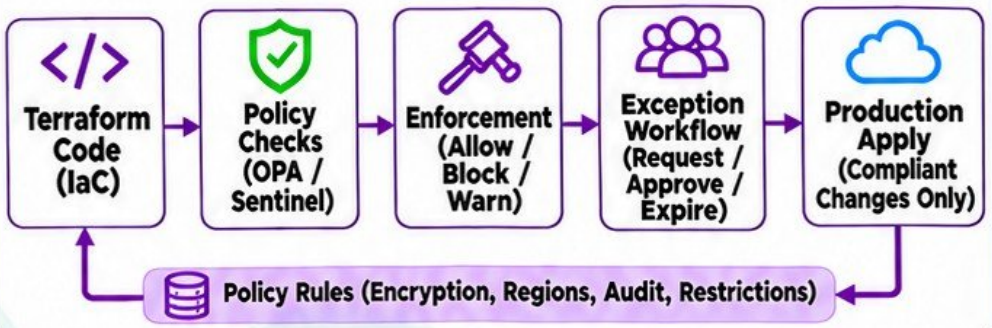
Build a Terraform-Based Policy Enforcement Platform

Project 23: Enforce encryption, approved regions, mandatory audit logging, resource restrictions, and exception workflows using policy-as-code checks before infrastructure changes reach production.

Objective

Build a policy enforcement platform for Terraform that automatically validates infrastructure changes against security, compliance, and organizational requirements before production.

Architecture Overview (Policy-as-Code for Terraform)



Key Capabilities

- **Policy-as-Code:** Define compliance and security rules (OPA/Sentinel).
- **Encryption Enforcement:** Block resources without encryption (e.g., storage, databases).
- **Approved Regions:** Allow only approved regions for deployments.
- **Mandatory Audit Logging:** Ensure audit logs are enabled for key services.
- **Resource Restrictions:** Enforce allowed instance types, sizes, and service usage.
- **Exception Workflows:** Controlled, time-bound approvals with audit trail.

Example: Policy & Enforcement (Sentinel)

```
# Sentinel policy: Enforce encryption and region
import "tfplan/v2" as plan

approved_regions = ["us-east-1", "eu-west-1"]

main = rule {
  all plan.resources as r {
    (r.type startswith "aws_") implies {
      r.applied.region in approved_regions and
      r.applied.encryption == true
    }
  }
}
```

Compliant: All resources are encrypted and deployed to approved regions.

Core Policy Rules (Examples)

Policy	Enforcement Outcome
Encryption Required	Block if encryption is disabled
Approved Regions	Block if region is not in the allowlist
Audit Logging	Block if logging is not enabled
Resource Restrictions	Block unapproved instance types
Exceptions	Allow with approval + expiry period

Benefits

- Consistent and compliant infrastructure.
- Prevents misconfigurations and policy drift.
- Automated, repeatable, and auditable checks.
- Controlled exceptions with traceability.
- Higher security and regulatory alignment.

Senior Engineer Insight

Policy-as-code with Terraform turns compliance and security requirements into automatic, enforceable guardrails.

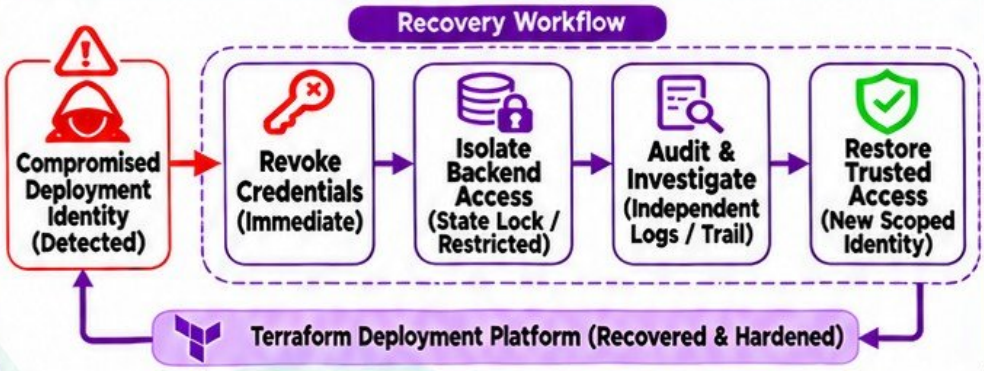
Recover From a Compromised Terraform Deployment Identity

Project 24: Build a recovery-ready deployment architecture with scoped identities, independent audit trails, emergency credential revocation, backend access isolation, and controlled restoration of trusted deployment access.

Objective

Design and implement a secure recovery process for a compromised Terraform deployment identity with minimal downtime and full traceability.

Architecture Overview (Identity Compromise Recovery)



Key Capabilities

- **Compromise Detection & Response:** Identify and respond to a breached deployment identity.
- **Credential Revocation:** Immediately revoke and invalidate compromised credentials.
- **Backend Access Isolation:** Restrict access to the Terraform state backend (e.g., S3, GCS, Azure Storage).
- **Independent Audit Trails:** Maintain separate, tamper-evident logs for deployments and admin actions.
- **Trusted Identity Restoration:** Create new, scoped, least-privilege deployment identity with MFA/short-lived access.
- **Controlled Recovery:** Validate and restore deployment access safely.

Example: Recovery Steps (AWS)

```
# 1. Revoke the compromised IAM access key
aws iam delete-access-key \
   --user-name terraform-deploy \
   --access-key-id AKIAEXAMPLE12345

# 2. Restrict access to the Terraform state backend (S3)
aws s3api put-bucket-policy \
   --bucket veriqta-terraform-state
   --policy file://deny-all-except-admins.json

# 3. Create a new, scoped deployment role
aws iam create-role \
   --role-name terraform-deploy-v2 \
   --assume-role-policy-document file://trust-policy.json

# 4. Update pipeline with the new role (OIDC / federation)
aws iam update-role-policy \
   --role-name terraform-deploy-v2
   --policy-name deploy-permissions
   --policy-document file://terraform-deploy.json
```

Recovery Checklist

Step	What to Do
Detect & Confirm	Validate the compromise (logs, alerts).
Revoke Access	Disable/delete compromised credentials.
Isolate Backend	Restrict state backend access (e.g., S3).
Audit & Investigate	Review logs and identify the root cause.
Restore Access	Create a new, scoped deployment identity.
Validate & Resume	Test a plan/apply with the new identity.

Benefits

- Rapid and controlled incident recovery.
- Prevents further unauthorized changes.
- Preserves a complete audit trail.
- Maintains backend and state integrity.
- Enforces least-privilege, scoped access.

Senior Engineer Insight

A strong recovery plan turns an identity compromise from a major outage into a controlled, auditable incident.